

Personal GitHub Setup and Workflow Guide

Safe use on a company-managed Windows laptop

A privacy-preserving, end-to-end procedure covering account creation, Git identity, SSH authentication, repository cloning, staging, committing, pushing, and troubleshooting.

Privacy: All usernames, email addresses, repository names, company paths, and screenshots have been replaced with generic examples.

Contents

1. Scope and safety model
2. Git, GitHub, identity, and authentication
3. Inspect the existing company configuration
4. Create a personal GitHub account and repository
5. Generate a dedicated personal SSH key
6. Add the public key to GitHub
7. Test SSH authentication
8. Work around networks that block SSH port 22
9. Clone the repository without modifying SSH config
10. Configure repository-local Git identity
11. Add or modify files
12. Stage changes
13. Commit changes
14. Configure repository-local SSH for future pushes
15. Push commits to GitHub
16. Empty folders and .gitkeep
17. Common failures and corrected procedures
18. Daily workflow checklist
19. Security and privacy recommendations
20. Command reference

1. Scope and safety model

This guide describes how to use a personal GitHub account from a Windows computer that already has company Git settings. The central design rule is isolation: preserve company-wide settings and place personal settings only where they are required.

- Do not overwrite the existing global Git username or email.
- Do not replace company authentication settings.
- Use a separate SSH key pair for the personal GitHub account.

- Use repository-local Git configuration for personal repositories.
- Avoid modifying the shared SSH config file when company policy or future use is uncertain.

Important: Git author identity and SSH authentication are separate mechanisms. A successful SSH login does not automatically set the commit author name or email.

2. Git, GitHub, identity, and authentication

2.1 Git

Git is a distributed version control system. It records snapshots of tracked files as commits and manages branches, history, differences, and synchronization.

2.2 GitHub

GitHub hosts Git repositories on remote servers and adds collaboration features such as pull requests, issues, reviews, and access control.

2.3 Commit identity

Every commit stores an author name and email. Git normally reads these values from configuration.

```
git config user.name "Personal Developer"
git config user.email "personal@example.com"
```

2.4 SSH authentication

SSH proves that the computer possesses a private key corresponding to a public key registered with GitHub. The private key stays on the computer; only the public key is uploaded.

3. Inspect the existing company configuration

Before changing anything, inspect the active Git configuration and the repository remote.

```
git config --list --show-origin
```

This displays every applicable setting and the file from which it was read.

```
git remote -v
```

This shows whether a repository uses HTTPS or SSH and identifies its fetch and push destinations.

Safe interpretation: If the global `user.name` and `user.email` contain company values, leave them unchanged. Personal repositories can override them locally.

4. Create a personal GitHub account and repository

Create a personal GitHub account using a personal email address. Then create a repository using a generic example such as:

```
Repository name: embedded-learning-notes
```

- Choose Public or Private according to the content.
- Do not upload company code, documents, credentials, or proprietary information.
- A README can be created on GitHub, but it is optional.

5. Generate a dedicated personal SSH key

Generate a separate Ed25519 key pair for the personal account.

```
ssh-keygen -t ed25519 -C "Personal GitHub key"
```

When prompted for a file name, use a distinct path such as:

```
C:/Users/WindowsUser/.ssh/id_ed25519_github_personal
```

This creates two files:

File	Purpose
id_ed25519_github_personal	Private key. Never upload or share it.
id_ed25519_github_personal.pub	Public key. Upload this to GitHub.

Passphrase: A passphrase protects the private key. If forgotten, it cannot be recovered; generate a new key pair and replace the old public key on GitHub.

6. Add the public key to GitHub

Display the public key:

```
cat ~/.ssh/id_ed25519_github_personal.pub
```

Copy the complete single-line output, then open GitHub Settings > SSH and GPG keys > New SSH key. Add a descriptive title such as:

```
Personal Windows Laptop
```

Paste only the public-key line. Never paste the private key.

7. Test SSH authentication

First test the standard GitHub SSH endpoint:

```
ssh -i ~/.ssh/id_ed25519_github_personal -T git@github.com
```

On the first connection, SSH may ask whether the host fingerprint should be trusted. Verify that the hostname is GitHub, then type:

```
yes
```

Successful result: A successful GitHub test reports that authentication succeeded and that GitHub does not provide shell access.

8. Work around networks that block SSH port 22

Some company or institutional networks block standard SSH on TCP port 22. GitHub provides an alternate SSH endpoint on port 443.

```
ssh -i ~/.ssh/id_ed25519_github_personal -T -p 443 git@ssh.github.com
```

Why the hostname changes: The standard endpoint is github.com on port 22. GitHub's SSH-over-443 endpoint is ssh.github.com on port 443.

Successful result: The command should authenticate to the personal GitHub account while still stating that shell access is unavailable.

9. Clone the repository without modifying SSH config

To avoid editing ~/.ssh/config, supply a temporary SSH command only for cloning.

```
GIT_SSH_COMMAND="ssh -i ~/.ssh/id_ed25519_github_personal" git clone  
ssh://git@ssh.github.com:443/PersonalUser/embedded-learning-notes.git
```

In Git Bash, the command can be split across lines with a trailing backslash:

```
GIT_SSH_COMMAND="ssh -i ~/.ssh/id_ed25519_github_personal" \  
git clone ssh://git@ssh.github.com:443/PersonalUser/embedded-learning-notes.git
```

The temporary GIT_SSH_COMMAND value applies only to that command. It does not alter global Git configuration or SSH configuration.

Correct URL format: For SSH over port 443, use ssh://git@ssh.github.com:443/USER/REPOSITORY.git. The normal GitHub-generated shorthand git@github.com:USER/REPOSITORY.git uses the standard port 22.

10. Configure repository-local Git identity

Enter the cloned repository and inspect the effective identity before changing it:

```
cd embedded-learning-notes  
git config --show-origin --get user.name  
git config --show-origin --get user.email
```

If the output points to the global company configuration, create a local override:

```
git config --local user.name "Personal Developer"  
git config --local user.email "personal@example.com"
```

Verify that the origin is now .git/config:

```
git config --show-origin --get user.name  
git config --show-origin --get user.email
```

Result: Only this repository uses the personal identity. Company repositories continue to inherit the company global identity.

11. Add or modify files

Create new files or edit existing ones using an editor. Then inspect the working tree:

```
git status
```

Typical states include:

- Untracked file: a new file Git has never recorded.
- Modified file: a tracked file whose content changed.
- Deleted file: a tracked file removed from the working directory.

12. Stage changes

Staging selects the exact content that will be included in the next commit.

```
git add README.md
```

To stage all intended changes in the current repository:

```
git add .
```

Verify the staging area:

```
git status
```

Expected result: Staged files appear under 'Changes to be committed'.

13. Commit changes

Create a commit from the staged snapshot:

```
git commit -m "Add initial documentation structure"
```

Confirm the latest commit and its author:

```
git log -1 --format="Commit: %h%nAuthor: %an <%ae>%nMessage: %s"
```

Important: A commit remains local until it is pushed. GitHub does not receive it automatically.

14. Configure repository-local SSH for future pushes

The temporary SSH override used during clone is not saved. Configure this repository so future fetch, pull, and push operations use the personal key.

```
git config --local core.sshCommand "ssh -i C:/Users/WindowsUser/.ssh/id_ed25519_github_personal -o IdentitiesOnly=yes"
```

Ensure the remote uses GitHub's port-443 endpoint:

```
git remote set-url origin ssh://git@ssh.github.com:443/PersonalUser/embedded-learning-notes.git
```

Verify both settings:

```
git config --local --get core.sshCommand  
git remote -v
```

Isolation: Both values are stored inside the repository's `.git/config` and do not affect other repositories.

15. Push commits to GitHub

Push the current branch:

```
git push
```

For the first push of a new local branch, Git may require an upstream mapping:

```
git push -u origin main
```

The `-u` option records `origin/main` as the upstream branch. Later pushes can use simply `git push`.

Successful result: Git reports objects being written and updates the remote branch. Refreshing the GitHub repository page shows the new commit and files.

16. Empty folders and `.gitkeep`

Git tracks files and file content, not directory objects. Therefore, an empty folder does not appear in a commit or on GitHub.

To preserve a planned directory, create a placeholder file:

```
New-Item docs/.gitkeep -ItemType File
```

Or in Git Bash:

```
touch docs/.gitkeep
```

Then stage, commit, and push it:

```
git add docs/.gitkeep  
git commit -m "Add documentation directory"  
git push
```

Convention: `.gitkeep` is not a special Git command or built-in feature. It is simply a conventional placeholder filename.

17. Common failures and corrected procedures

17.1 Permission denied (publickey)

```
git@ssh.github.com: Permission denied (publickey).
```

Likely causes include the wrong private key, an unexpanded key path, a missing GitHub public key, or SSH offering another key first.

Correct the repository-local command using a full Windows path:

```
git config --local core.sshCommand "ssh -i C:/Users/WindowsUser/.ssh/id_ed25519_github_personal -o IdentitiesOnly=yes"
```

Test the exact endpoint:

```
ssh -i C:/Users/WindowsUser/.ssh/id_ed25519_github_personal -o IdentitiesOnly=yes -T -p 443 git@ssh.github.com
```

17.2 Could not read from remote repository

fatal: Could not read from remote repository.

Check the remote and authentication separately:

```
git remote -v  
ssh -T -p 443 git@ssh.github.com
```

If the remote still uses github.com shorthand while the network blocks port 22, replace it with the explicit port-443 URL.

17.3 Clone worked but push failed

This happens when GIT_SSH_COMMAND was used only during clone. That environment variable was temporary and was not retained.

Store core.sshCommand locally in the cloned repository as shown in Section 14.

17.4 Git shows the company author in a personal repository

The repository is inheriting the global configuration. Set user.name and user.email locally before committing.

```
git config --local user.name "Personal Developer"  
git config --local user.email "personal@example.com"
```

17.5 A new folder did not appear on GitHub

The folder was empty. Add a real file such as README.md or a placeholder such as .gitkeep.

17.6 Wrong commit author was already recorded

If the incorrect commit has not been pushed, correct the local identity and amend the latest commit:

```
git config --local user.name "Personal Developer"  
git config --local user.email "personal@example.com"  
git commit --amend --reset-author --no-edit
```

Caution: Do not rewrite shared or already-pushed history unless you understand the effect on collaborators.

18. Daily workflow checklist

Use this repeatable sequence whenever working in the personal repository:

```
cd path/to/personal-repository
git status
git remote -v
git pull
# Edit files
git status
git diff
git add <files>
git status
git commit -m "Clear description of the change"
git log -1 --oneline
git push
```

Recommended habit: Before every push, confirm the current directory, branch, remote, and staged content.

19. Security and privacy recommendations

- Never commit passwords, tokens, private keys, certificates, or company secrets.
- Never upload the private SSH key. Only upload the .pub file.
- Use a different SSH key for each device when practical.
- Remove a device's public key from GitHub immediately if the device is lost.
- Keep personal projects in a separate workspace from company repositories.
- Use a repository-specific .gitignore for build outputs, IDE files, logs, and secrets.
- Review git status and git diff before every commit.
- Confirm that personal work is permitted on the company-managed device and network.

Example .gitignore entries for embedded development:

```

# Build outputs
build/
out/
*.o
*.obj
*.elf
*.hex
*.bin
*.map

# IDE and editor files
.vscode/
.idea/
*.user

# Logs and temporary files
*.log
*.tmp

# Secrets
.env
*.pem
*.key

```

20. Command reference

Purpose	Command
Inspect all configuration	git config --list --show-origin
Inspect effective name	git config --show-origin --get user.name
Inspect effective email	git config --show-origin --get user.email
Set local name	git config --local user.name "Personal Developer"
Set local email	git config --local user.email "personal@example.com"
Show remotes	git remote -v
Change origin URL	git remote set-url origin <URL>
Show status	git status
Show unstaged differences	git diff
Show staged differences	git diff --staged
Stage one file	git add <file>
Stage all current changes	git add .
Commit	git commit -m "Message"
Show compact history	git log --oneline --graph --decorate
Push	git push
Pull	git pull
Test GitHub SSH on 443	ssh -T -p 443 git@ssh.github.com

Final verified workflow

The completed setup keeps the company configuration intact while allowing a personal GitHub repository to use a dedicated SSH key, a repository-local author identity, and a repository-local SSH command. The complete flow is:

```
Create personal GitHub account
|
Create repository
|
Generate dedicated SSH key pair
|
Upload public key to GitHub
|
Test SSH authentication
|
Use ssh.github.com:443 when port 22 is blocked
|
Clone with a temporary SSH override
|
Set local user.name and user.email
|
Create or modify files
|
git status
|
git add
|
git commit
|
Set repository-local core.sshCommand
|
git push
```

Outcome: Personal and company Git usage remain isolated, provided each command is executed from the intended repository and no proprietary material is copied into the personal repository.